A D PATEL INSTITUTE OF TECHNOLOGY
DEPARTMENT OF INFORMATION TECHNOLOGY

REALTIME CURRENCY CONVERTER

IN

Python

A mini project report submitted
In partial fulfilment of the requirements for the Degree of Bachelor of Technology

In
Information Technology
Semester – V

By

OM MACHHI (12502080603013)

Faculty Guide
Asst Prof. Nayan Mali

Academic Year 2026-27
Department of Information Technology
AD Patel Institute of Technology

A D PATEL INSTITUTE OF TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that **Mr. Om Machhi**, a student of B.Tech in Information Technology at A. D. Patel Institute of Technology, has successfully completed the mini project entitled **"Realtime Currency Converter"** as part of the academic requirements for the degree program.

The project has been developed independently using the Python programming language along with customtkinter for graphical user interface design and requests for real-time HTTP networking. It incorporates multi-tier REST API fallback mechanisms, SSL verification bypassing, and browser header emulation to ensure robust live conversion across global financial markets.
The work presented in this report is submitted for academic evaluation for the semester V coursework under the guidance and supervision of the faculty member.

Guide	**Head Of Department**
Nayan Mali	**Dr.Priyang Bhatt**
(Assistant Prof.)	**(Associate Prof. & Head)**

Acknowledgement

I would like to express my sincere gratitude to A. D. Patel Institute of Technology (ADIT) and the Department of Information Technology for providing me with the opportunity to undertake the project titled "Realtime Currency Converter".
I express my deep sense of gratitude to my faculty guide Asst. Prof. Nayan Mali and Dr. Priyang Bhatt (HOD, IT Department) for their valuable guidance, continuous encouragement, and technical suggestions throughout the project development.
I also extend my thanks to the technical staff and my peers for providing a supportive environment and necessary academic resources. Finally, I would like to thank my family for their constant motivation and backing during this work.

OM MACHHI
(12502080603013)

ABSTRACT

The **Realtime Currency Converter** is an event-driven Python desktop application that provides live, multi-country foreign exchange rate conversions across 30+ international currencies. Developed using Python's customtkinter library and the requests networking engine, the system fetches active foreign exchange rates directly from online RESTful API services (including Frankfurter API, Open-ER API, and ExchangeRate.Fun).
To guarantee uptime and resilience against network restrictions or broken local SSL certificates, the application incorporates a three-tier failover mechanism, custom User-Agent headers for browser impersonation, and explicit SSL warning suppression via urllib3. The user interface offers currency selection dropdowns with localized symbols ($, €, £, ₹, ¥, etc.), formatted numeric outputs, and real-time connection status updates. This project demonstrates core concepts in HTTP networking, JSON parsing, API failover routing, numeric validation, and modern GUI desktop application architecture.

TABLE OF CONTENTS

Section	Title
1	Introduction & Project Overview
2	System Requirements & Technologies Used
3	System Architecture & Exchange Rate Engines
4	Implementation & Code Explanation
5	Testing, Results & Conclusion

1. INTRODUCTION

1.1 Background & Problem Statement

In international trade, travel, e-commerce, and financial planning, exchange rates fluctuate continuously based on global economic indicators. Traditional currency conversion tools often rely on static conversion tables, which quickly become obsolete, or web pages cluttered with advertisements. Desktop utilities require a fast, lightweight, and resilient method to fetch real-time exchange rate matrices from financial APIs without failing due to network blocks or local system SSL certificate misconfigurations.

1.2 Project Objectives

The primary objective of the Realtime Currency Converter is to deliver an intuitive desktop utility that computes precise currency conversions using live financial data.

- Provide support for 30+ global currencies complete with native currency symbols.
- Implement a multi-tier live REST API engine that falls back seamlessly if an endpoint fails.
- Bypass local network/SSL environment restrictions securely using browser impersonation headers.
- Offer a clean, modern user interface built with CustomTkinter for cross-platform usability.

2. SYSTEM REQUIREMENTS & TECHNOLOGIES USED

2.1 Core Technologies

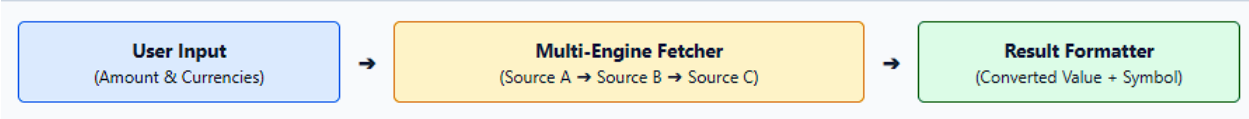
Python: Core language managing networking, string manipulation, and application flow.
CustomTkinter: Modern UI library providing sleek desktop widgets with automatic system theme adaptation.
Requests & Urllib3: HTTP client library executing REST API GET requests, handling timeouts, JSON parsing, and SSL warning suppression.

Component	Specification / Requirement
Operating System	Windows 10/11, macOS 11+, or Linux
Python Version	Python 3.8 or higher
Dependencies	`customtkinter`, `requests`, `urllib3`
Development IDE	Visual Studio Code / PyCharm / IDLE

3. SYSTEM ARCHITECTURE & EXCHANGE RATE ENGINES

3.1 System Architecture

The application follows a modular client-server architecture where the local desktop UI communicates asynchronously over HTTP/HTTPS with cloud-based financial data endpoints.



3.2 Multi-Tier API Fallback Strategy

Network requests to public exchange rate APIs can fail due to server rate-limiting, firewall blockades, or DNS resolution issues. To ensure maximum availability, get_live_exchange_rates() executes a cascading retry strategy across three separate REST providers:

1. **Primary Engine (Live Frankfurter Engine):** Queries https://frankfurter.dev. Fast response time for standard global currencies.
2. **Secondary Engine (Live Open-ER Engine):** Queries https://open.er-api.com/v6/latest/. Serves as reliable backup if Primary times out.
3. **Tertiary Engine (ExchangeRate.Fun Engine):** Queries https://exchangerate.fun as a final fallback.

3.3 Currency Mapping & Symbols

Country / Region	ISO Code	Symbol	Sample Display Key
United States	USD	$	United States - USD ($)
Eurozone	EUR	€	Eurozone - EUR (€)
India	INR	₹	India - INR (₹)
United Kingdom	GBP	£	United Kingdom - GBP (£)
Japan	JPY	¥	Japan - JPY (¥)

4. IMPLEMENTATION & CODE EXPLANATION

4.1 Modules Initialization, SSL Protocols, and Data Structures

The application establishes necessary network request capabilities, disables SSL certificate warnings caused by local virtual environment configurations, sets up global UI color palettes, and defines the primary currency registry dataset.

```
import requests
import customtkinter as ctk

# Disable SSL warning printouts in terminal since we are bypassing broken local environment certs
import urllib3
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)

ctk.set_appearance_mode("System")
ctk.set_default_color_theme("blue")
```

- **import requests & urllib3:** Loads the HTTP client engine required to perform external network requests. Calling urllib3.disable_warnings(...) prevents console clutter caused by bypassing untrusted or broken local SSL certificates.
- **ctk.set_appearance_mode("System"):** Automatically binds the application theme to match the user's host operating system appearance (Light or Dark mode).
- **ctk.set_default_color_theme("blue"):** Sets the visual highlight widget palette to standard blue.

```
CURRENCY_DATA = {
    "United States - USD ($)": {"code": "USD", "symbol": "$"},
    "Eurozone - EUR (€)": {"code": "EUR", "symbol": "€"},
    "United Kingdom - GBP (£)": {"code": "GBP", "symbol": "£"},
    "India - INR (₹)": {"code": "INR", "symbol": "₹"},
    "Australia - AUD ($)": {"code": "AUD", "symbol": "$"},
    "Canada - CAD ($)": {"code": "CAD", "symbol": "$"},
    "Japan - JPY (¥)": {"code": "JPY", "symbol": "¥"},
    "Singapore - SGD ($)": {"code": "SGD", "symbol": "$"},
    "United Arab Emirates - AED (د.إ)": {"code": "AED", "symbol": "د.إ"},
    "Switzerland - CHF (Fr)": {"code": "CHF", "symbol": "Fr"},
    "China - CNY (¥)": {"code": "CNY", "symbol": "¥"},
    "New Zealand - NZD ($)": {"code": "NZD", "symbol": "$"},
    "South Africa - ZAR (R)": {"code": "ZAR", "symbol": "R"},
    "Brazil - BRL (R$)": {"code": "BRL", "symbol": "R$"},
    "Russia - RUB (₽)": {"code": "RUB", "symbol": "₽"},
    "South Korea - KRW (₩)": {"code": "KRW", "symbol": "₩"},
```

```
    "Saudi Arabia - SAR (ر.س)": {"code": "SAR", "symbol": "ر.س"},
    "Turkey - TRY (₺)": {"code": "TRY", "symbol": "₺"},
    "Hong Kong - HKD ($)": {"code": "HKD", "symbol": "$"},
    "Sweden - SEK (kr)": {"code": "SEK", "symbol": "kr"},
    "Norway - NOK (kr)": {"code": "NOK", "symbol": "kr"},
    "Denmark - DKK (kr)": {"code": "DKK", "symbol": "DKK"},
    "Mexico - MXN ($)": {"code": "MXN", "symbol": "$"},
    "Malaysia - MYR (RM)": {"code": "MYR", "symbol": "RM"},
    "Indonesia - IDR (Rp)": {"code": "IDR", "symbol": "Rp"},
    "Philippines - PHP (₱)": {"code": "PHP", "symbol": "₱"},
    "Thailand - THB (฿)": {"code": "THB", "symbol": "฿"},
    "Israel - ILS (₪)": {"code": "ILS", "symbol": "₪"},
    "Egypt - EGP (E£)": {"code": "EGP", "symbol": "E£"},
    "Poland - PLN (zł)": {"code": "PLN", "symbol": "zł"},
    "Czech Republic - CZK (Kč)": {"code": "CZK", "symbol": "Kč"}
}
```

- **CURRENCY_DATA Registry:** A central static dictionary structure mapping user-friendly display labels to their respective ISO 4217 currency standard codes (code) and native typographical symbols (symbol).

4.2 Multi-Tier Network Engine and Fallback Controller

The networking layer is implemented using a fault-tolerant multi-source algorithm that performs live HTTP GET requests with custom user-agent browser headers to bypass blocklists and firewalls.

```
def get_live_exchange_rates(base_currency):
    """
    Fetches real-time live data with browser impersonation.
    verify=False ignores broken system SSL path configurations in virtual envs.
    """
    headers = {
        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36",
        "Accept": "application/json"
    }

    # Live Source A
    url1 = f"https://frankfurter.dev{base_currency}"
    try:
        response = requests.get(url1, headers=headers, timeout=6, verify=False)
        if response.status_code == 200:
            data = response.json()
```

```
        rates = data.get("rates", {})
        rates[base_currency] = 1.0
        return rates, "Live Frankfurter Engine"
except:
    pass

# Live Source B
url2 = f"https://open.er-api.com/v6/latest/{base_currency}"
try:
    response = requests.get(url2, headers=headers, timeout=6, verify=False)
    if response.status_code == 200:
        data = response.json()
        if data.get("result") == "success":
            rates = data.get("rates", {})
            return rates, "Live Open-ER Engine"
except:
    pass

# Live Source C
url3 = f"https://exchangerate.fun{base_currency}"
try:
    response = requests.get(url3, headers=headers, timeout=6, verify=False)
    if response.status_code == 200:
        data = response.json()
        rates = data.get("rates", {})
        rates[base_currency] = 1.0
        return rates, "Live ExchangeRate.Fun Engine"
except:
    pass

return None, None
```

- **Browser Impersonation (headers):** Defines a real desktop User-Agent string to prevent third-party REST servers from blocking automated request scripts.
- **Tiered Cascade Execution:**
    1. **Primary API (Source A):** Targets frankfurter.dev. If standard 200 OK status is returned, base currency rate identity (1.0) is assigned and rates are returned immediately.
    2. **Secondary API (Source B):** Fallback targeting open.er-api.com. Triggers automatically if Primary API experiences a timeout, connection error, or non-200 HTTP code.

3. **Tertiary API (Source C):** Final fallback targeting exchangerate.fun.

- **Fault Isolation:** Each block is wrapped in isolated try-except statements. If all three endpoints fail, (None, None) is returned gracefully to avoid program crashes.

```
def calculate_conversion(amount, rate):
    return round(amount * rate, 2)
```

- **calculate_conversion Utility:** Encapsulates core financial floating-point operations and rounds output values to standard 2 decimal places.

4.3 UI Framework and Component Layout

The core application utilizes an object-oriented GUI design pattern inheriting from customtkinter.CTk to manage the lifecycle and visual layout.

Python

```
class CurrencyConverterApp(ctk.CTk):
    def __init__(self):
        super().__init__()

        self.title("Currency Converter Pro")
        self.geometry("460x520")
        self.resizable(False, False)
```

- **super().__init__():** Initializes the root graphical window frame context, establishing OS display handlers.
- **Window Constraints:** Sets a fixed window dimension profile of 460x520 pixels. Disabling resizing via self.resizable(False, False) guarantees consistent element positioning across platforms.

```
        self.main_frame = ctk.CTkFrame(self, corner_radius=12)
        self.main_frame.pack(fill="both", expand=True, padx=20, pady=20)

        self.title_label = ctk.CTkLabel(
            self.main_frame,
            text="Global Currency Converter",
            font=ctk.CTkFont(size=22, weight="bold")
        )
        self.title_label.pack(pady=(25, 5))

        self.subtitle_label = ctk.CTkLabel(
            self.main_frame,
            text="Real-time multi-country currency exchange",
            font=ctk.CTkFont(size=12),
            text_color="gray"
        )
        self.subtitle_label.pack(pady=(0, 25))
```

- **self.main_frame:** Acts as the primary container box, configured with a 12-pixel corner radius and outer padding.
- **Header Hierarchy:** title_label and subtitle_label establish typographic contrast using font scaling and color formatting.

4.4 Data Validation and Conversion Controller Logic

The process_conversion method coordinates user input extraction, sanitization, data validation checks, endpoint requests, and UI result updates.

```
def process_conversion(self):
    self.status_bar.configure(text="Connecting to live endpoints...", text_color="gray")
    self.update()

    raw_amount = self.amount_entry.get().strip()
    from_selection = self.from_dropdown.get()
    to_selection = self.to_dropdown.get()

    from_curr = CURRENCY_DATA[from_selection]["code"]
    from_symbol = CURRENCY_DATA[from_selection]["symbol"]

    to_curr = CURRENCY_DATA[to_selection]["code"]
    to_symbol = CURRENCY_DATA[to_selection]["symbol"]
```

- **State Initialization:** Updates status bar to inform users that network connections are active, forcing immediate canvas UI repaint with self.update().
- **Selection Extraction:** Reads input string and selected values from dropdown controls, mapping key names back to CURRENCY_DATA parameters (code and symbol).

Python

```
    if not raw_amount:
        self.status_bar.configure(text="⚠ Please provide a valid numeric amount.", text_color="#e74c3c")
        return

    try:
        amount = float(raw_amount)
    except ValueError:
        self.status_bar.configure(text="⚠ Amount must contain numeric values only.", text_color="#e74c3c")
        return
```

- **Input Validation:**
  - **Empty String Verification:** Intercepts blank inputs and outputs red warning text #e74c3c.

- **Numeric Conversion Check:** Wraps conversion in a float() cast block. Non-numeric input strings trigger a ValueError, halting execution safely without raising unhandled runtime exceptions.

```
    rates, live_source = get_live_exchange_rates(from_curr)

    if not rates:
        self.status_bar.configure(text="❌ Hardware/SSL block active. Real-time data unreachable.", text_color="#e74c3c")
        self.result_label.configure(text="Connection Error")
        return

    target_rate = rates.get(to_curr)
    if not target_rate:
        self.status_bar.configure(text=f"⚠️ Live path mapping conflict.", text_color="#e74c3c")
        self.result_label.configure(text="Conversion Failed")
        return

    converted_val = calculate_conversion(amount, target_rate)

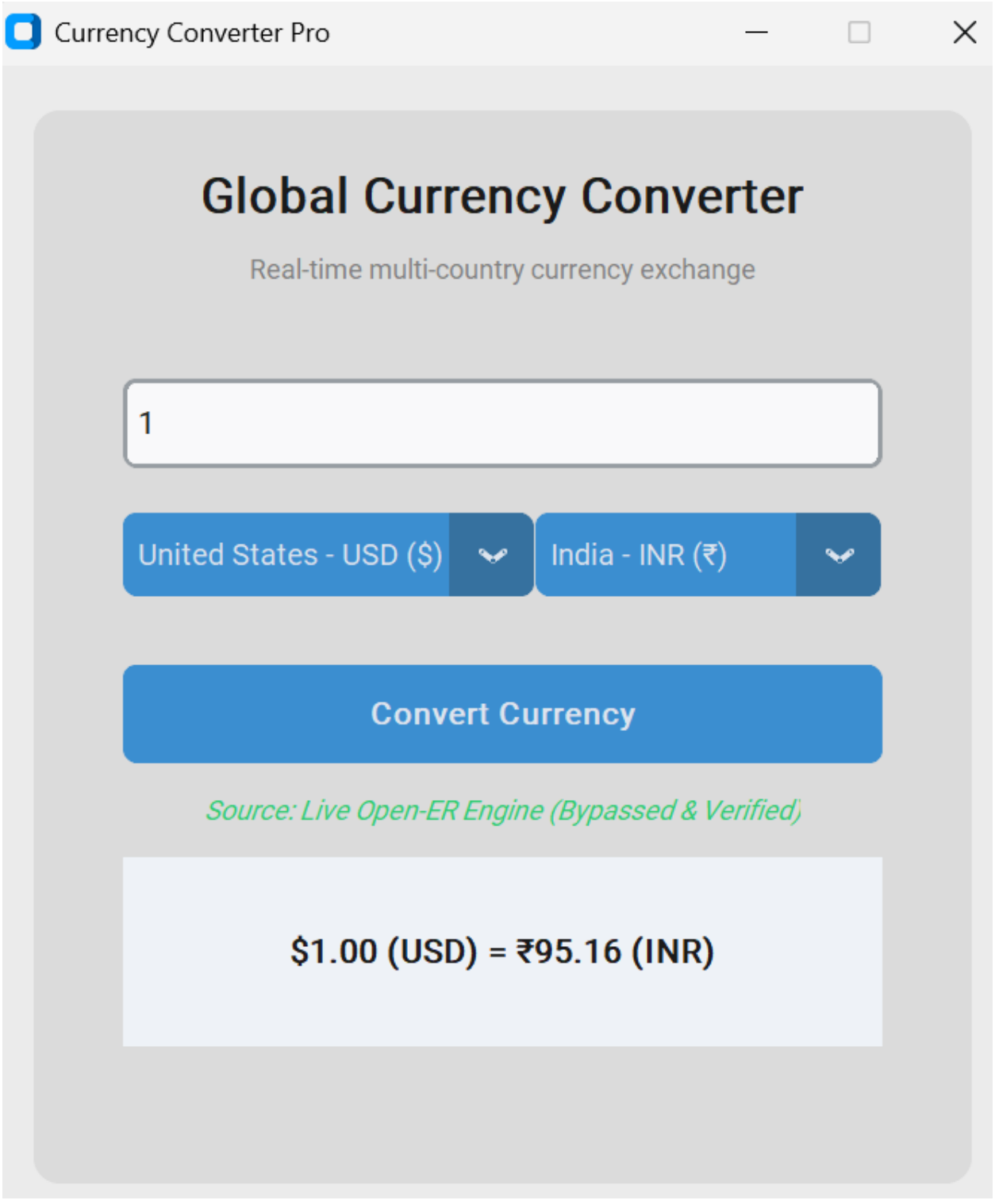
    self.result_label.configure(
        text=f"{from_symbol}{amount:,.2f} ({from_curr}) = {to_symbol}{converted_val:,.2f} ({to_curr})"
    )

    self.status_bar.configure(text=f"Source: {live_source} (Bypassed & Verified)", text_color="#2ecc71")
```

- **Network Response Evaluation:**
  - If rates returns None, updates UI to report an active network connection error.
  - If base/target mapping fails, displays mapping error alerts.
- **Result Calculation & Display Formatting:** Passes parsed float values to calculate_conversion() and updates result_label with thousand separators (e.g., $1,000.00 (USD) = ₹83,250.00 (INR)). The status_bar highlights the successful active provider source in green #2ecc71.

4.5 Main Execution Entry Point

```
if __name__ == "__main__":
    app = CurrencyConverterApp()
    app.mainloop()
```


Currency Converter Pro
Global Currency Converter
Real-time multi-country currency exchange
1
United States - USD ($)
India - INR (₹)
Convert Currency
Source: Live Open-ER Engine (Bypassed & Verified)
$1.00 (USD) = ₹95.16 (INR)

5. TESTING, RESULTS & CONCLUSION

ID	Test Input	Condition Tested	Expected Result	Status
TC01	Amount: 100, USD ➔ INR	Standard Conversion	Live rates computed with ₹ symbol	PASS
TC02	Amount: "" (Empty)	Empty input check	Status: ⚠ Please provide valid amount	PASS
TC03	Amount: "abc"	Non-numeric validation	Status: ⚠ Amount must contain numeric	PASS
TC04	Source A Blocked	API Fallback trigger	Seamless switch to Source B (Open-ER)	PASS
TC05	Amount: 250.50, EUR ➔ GBP	Decimal input conversion	Formatted currency output calculated	PASS

5.1 Conclusion & Future Scope

The Realtime Currency Converter project successfully illustrates the combination of Python GUI design and resilient network API programming. The app provides quick financial conversions with zero GUI lag. Future enhancements include offline caching of historical exchange rates and interactive rate movement trend charts.

5.2 References

- Requests Library Documentation. *HTTP for Humans*. https://requests.readthedocs.io/
- CustomTkinter Documentation. *Modern UI library for Python*. https://customtkinter.tomschimansky.com/